# Tolerant frames

A modal account of fact preservation without ownership

TScert working note

11 November 2026

**Abstract**

TypeScript programs are not written against an ownership discipline, and TScert imposes none; it must still decide which facts about the heap survive a step. This note gives that reasoning a surface language. Its one new connective is a modality $\Box_\Phi F$, "$F$ holds here and after any change bounded by the effect $\Phi$", which stands where the separating conjunction stands in the frame rule. A fact enters the modality for one of three reasons: the effect is disjoint from what the fact reads; the effect changes what the fact reads only in a direction the fact tolerates; or the invariant annotations on a type fix what the fact reads. Eleven patterns from a TypeScript code base become one-line annotations, and four more show what the language still lacks. Every statement carries one of two marks. [Lean] names the theorem of the TScert development that proves the statement's content, and gives that theorem's scope; where the theorem is narrower than the statement, the mark says how. [conj.] marks a conjecture.

## 1 The problem

**Example 1.** `for (let j = from; j < to; j++) out[j] = xs[j];` The read is in bounds if $to \le |xs|$ holds at each iteration. The array `out` may be an alias of `xs`. Since an element write never shortens an array, the bound survives.

**Example 2.** `while (i < flags.length) { readRuns(runs); s += flags[i++]; }` The callee calls `runs.openBrackets.splice(...)`, which may shorten that array. The bound $i < |flags|$ survives the call because the array the callee shortens is not `flags`, and because the field `openBrackets` keeps denoting that array.

Example 2 is a frame problem. Example 1 is not: no disjointness is available, and none is required. What the two share is that the step's *effect* is known and the fact is closed under it. The language below makes that the primitive.

## 2 What exists: variables, locations, stores

**Variables and heap objects.** Let $Var$ be the set of variables, ranged over by $x$, $y$, and $Val$ the set of values, ranged over by $v$. An environment $\rho : Var \rightharpoonup Val$ maps variables to values. A variable holds a scalar or an object reference. A binding introduces a fresh variable; existing variables retain their values. Object fields and array elements hold their contents in the heap, where writes change them. A private cell is an object with one field, `value`: allocating `r = {value: a}` returns its reference, `y = r.value` reads its contents, and `r.value = a` writes them.

**Lowering.** Lowering maps each TypeScript binding to a value variable or a private cell and translates its reads and assignments to value bindings or field operations. It uses cells for bindings whose mutation must be shared with closures or whose initialization requires checked access; assignments to bindings held directly as values introduce fresh variables, connected by the edge renamings of Section 7.

**Definition 3** (Commands)**.** Programs name intermediate results explicitly. An *atom* $a$ is a variable or a scalar literal. A *step* is one of:

| | |
|---|---|
| `y = a`, `y = a + a'` and the other operators | a binding |
| `y = x.f`, `x.f = a` | a read and a write of a field |
| `y = x[a]`, `x[a] = a'` | a read and a write of an element |
| `y = {f: a, ...}`, `y = [a, ...]` | an allocation |
| `y = x.m(x, ...)` | a call of a native method, such as `push` |
| `y = f(x, ...)` | a call of a function |

Each step binds at most the one fresh variable `y`. A *command* $c$ is a step, a sequence `c; c`, a choice `if (a) c else c`, a loop `loop c`, which repeats $c$ until $c$ exits, a handler `try c catch (x) c`, or an exit `return a`, `throw a` or `break`.

Example: the source statement `out[j] = xs[j]` is the command `v = xs[j]; out[j] = v`. Each intermediate result is named before it is used. Private cells use the object allocation and field operations of this same syntax. The allocation typing records the storage role of each field in the world (Definition 4); a cell's `value` field has role cell, an ordinary property role field.

Effects bound heap changes. Variables retain their values, so a fact about variables alone, such as $i < n$, persists.

**Locations.** A heap $h$ is a finite map from locations, ranged over by $l$, to contents. Locations are natural numbers, and the domain of $h$ is the initial segment $\{0, \ldots, |h| - 1\}$: an allocation extends it, and no location is ever removed. The locations of a heap $h$ are of five kinds, by what allocated them: field, cell, elem, store and id. Write $Loc_\kappa$ for the set of its locations of kind $\kappa$; the five sets partition its domain, $\mathrm{dom}\, h = Loc_{\mathsf{field}} \uplus Loc_{\mathsf{cell}} \uplus Loc_{\mathsf{elem}} \uplus Loc_{\mathsf{store}} \uplus Loc_{\mathsf{id}}$. A location's kind is fixed for its life. Each kind belongs to one of two classes, *value* or *store*; effects refer to locations by class.

### Field location, kind field

One for each ordinary object property. It holds the current value of that property. After `const o = { n: 1, m: 2 };` the object has two field locations, one for the key `n` holding 1 and one for the key `m` holding 2; `o.n = 3` changes what the first holds and leaves the second. Class: *value*.

### Cell location, kind cell

A location of private binding storage, including the fields of initialization markers. A binding cell uses such a location for its `value` field: `r = {value: 0}` has one, holding 0, and an identity location. The write `r.value = 1` changes the former to 1. Class: *value*.

### Element cell, kind elem

One for each present element of each array. It holds that element's current value. In JavaScript an array literal with nothing between two commas leaves that index empty: `const a = [10, , 30];` makes an array of length 3 with elements at the indices 0 and 2 and none at index 1. An index with no element is a *hole*. This array has two element cells, $l_0$ holding 10 and $l_2$ holding 30; the hole at index 1 has no location. `a[0] = 11` changes what $l_0$ holds. Class: *value*.

### Store location, kind store

One for each array. It holds the array's *store*: the sequence, in index order, of the array's element cells and holes. For `a` above the store is $\langle l_0, \circ, l_2 \rangle$. `a.push(40)` allocates an element cell $l_3$ and changes the store to $\langle l_0, \circ, l_2, l_3 \rangle$; `a.length = 1` changes it to $\langle l_0 \rangle$. Class: *store*.

A store location has one of two *forms*, fixed with its kind. An *array store* belongs to an ordinary array, and its length may change, as above. A *view store* belongs to a typed array, such as a `Uint8Array`. It lists the cells that hold what the typed array reads at each index, and its length is fixed: after `const b = new Uint8Array(4);` the view store of `b` has length 4 for the life of `b`. We write the forms array and $\mathsf{view}_N$, with $N$ the length.

### Identity location, kind id

One for each object, allocated with it. It holds `null`, and its contract admits `null` alone. Two objects are the same object when they have the same identity location. After `const o = { n: 1, m: 2 };` the object has one identity location beside its two field locations. Class: *value*.

So a heap maps a location to a value or to a store: $h(l) \in \mathit{Val}$ for $l \in Loc_{\mathsf{field}} \cup Loc_{\mathsf{cell}} \cup Loc_{\mathsf{elem}} \cup Loc_{\mathsf{id}}$, and $h(l) \in \mathit{Store}$ for $l \in Loc_{\mathsf{store}}$. Here $\mathit{Store} = (\mathit{Loc} \uplus \{\circ\})^*$ is the set of finite sequences whose entries are locations or

the hole $\circ$, and $Loc = \mathbb{N}$. For a store $\sigma = \langle c_0, \ldots, c_{n-1} \rangle$ we write $|\sigma| = n$ for its length and $cells(\sigma) = \{c_i \mid c_i \neq \circ\}$ for the set of locations it lists. In one line, $h : Loc \rightharpoonup Val \uplus Store$.

A value in *Val* is a scalar or an object. A function is an object that also carries code. A *scalar* is `null`, `undefined`, a number, a string, a boolean or a bigint; $Scalar \subseteq Val$ is the set of scalars and $Obj \subseteq Val$ the set of objects. An object is a finite map $v : Key \rightharpoonup Loc_{\mathsf{field}} \cup Loc_{\mathsf{cell}}$ from its own keys to field locations, together with an identity location in $Loc_{\mathsf{id}}$; both are fixed when the object is allocated. The read of a field $f$ of $v$ in a heap $h$ is $h(v(f))$. What changes is $h$ at the locations $v(f)$. An array is an object with one distinguished field, whose location $s \in Loc_{\mathsf{store}}$ is the array's store location. Its *store* is the sequence $h(s) = \langle c_0, \ldots, c_{n-1} \rangle \in Store$, where $c_i \in Loc_{\mathsf{elem}}$ if index $i$ is present and $c_i = \circ$ if it is a hole. The array's length is $n$, and its element at a present index $i$ is $h(c_i)$. For `const a = [10, , 30];` the store is $h(s) = \langle l_0, \circ, l_2 \rangle$, the length is 3, $h(l_0) = 10$ and $h(l_2) = 30$.

**Why distinguish fields from cells?** The kinds carry a separation fact. If `r.value` has kind cell and `o.value` has kind field, they denote different locations: one location has one kind. Thus, from `r.value = 0`, a write `o.value = 1` preserves `r.value = 0`. Merging the two kinds would admit `r = o`, making that conclusion false without a separate separation premise. The distinction supplies this reasoning at an interface, where the allocation history of the two objects is unavailable; both kinds use the same field-read and field-write operations.

**A whole heap.** Take an ordinary object `o = {n: 1, m: 2}`, a private cell `r = {value: 0}`, an array `a = [10, , 30]`, and a function object `inc` whose code increments `r.value`. Number their locations as follows:

| location | kind | holds | |
|---|---|---|---|
| 0 | field | 1 | the field `n` of `o` |
| 1 | field | 2 | the field `m` of `o` |
| 2 | cell | 0 | the field `value` of the private cell `r` |
| 3 | store | $\langle 4, \circ, 5 \rangle$ | the store of `a` |
| 4 | elem | 10 | the element `a[0]` |
| 5 | elem | 30 | the element `a[2]` |
| 6, 7, 8, 9 | id | `null` | the identity locations of `o`, `r`, `inc` and `a` |

Here $\mathrm{dom}\, h = \{0, \ldots, 9\}$, $Loc_{\mathsf{field}} = \{0, 1\}$, $Loc_{\mathsf{cell}} = \{2\}$, $Loc_{\mathsf{store}} = \{3\}$, $Loc_{\mathsf{elem}} = \{4, 5\}$ and $Loc_{\mathsf{id}} = \{6, 7, 8, 9\}$. The object `o` is the map $\{\mathtt{n} \mapsto 0, \mathtt{m} \mapsto 1\}$: `o.n` is read as $h(0)$ and `o.m` as $h(1)$. The cell `r` is the object $\{\mathtt{value} \mapsto 2\}$, so `r.value` is read as $h(2)$. The array `a` has store location 3. The environment $\rho$ holds `o`, `r`, `inc` and `a`. A call `inc()` changes $h(2)$ to 1 and leaves the other existing locations unchanged; `o.m = 7` changes $h(1)$ to 7 and leaves the others unchanged.

The steps of Definition 3 write as follows:

| operation | writes | class written |
|---|---|---|
| `y = a` | nothing in the heap | |
| `r.value = a`, `r` a private cell | the cell's contents location | *value* |
| `o.f = e` | the field location | *value* |
| `a[i] = e`, index present | the element cell | *value* |
| `a[i] = e` at a hole or past the end; `push` | the store (longer or filled), and a new cell | *store* |
| `pop`, `splice`, `a.length = n` | the store (possibly shorter) | *store* |

The *value* class is thus "contents": every location that holds something other than a store. The *store* class is "shape": which elements an array has and how many. A fact about a length or about presence reads a store; a fact about what a field or an element contains reads a value location.

**Which locations can change.** A location never moves or disappears; what changes is its contents. We say that a location $l$ *changes* between two heaps $h$ and $h'$ when $h'(l) \neq h(l)$. Every location can change, through any alias. What bounds the change is the location's *contract*: the set of contents the location may hold, fixed when the location is allocated. A field declared `n: number` has the contract "a number": `o.n = 2` is within it and `o.n = "a"` is outside it. TypeScript's `readonly` restricts one view of a location and makes no location immutable, since another alias may write.

A location is *immutable* exactly when its contract admits a single value. Three kinds of location are immutable. The first is a field declared `@stable`. This is an annotation of TScert, written in a comment on a `readonly` field of a record type: `type Runs = { /** @stable */ readonly openBrackets: number[] }`. It states that the field holds, for the object's whole life, the value the object was created with; the contract of the field's location

is that one value. Here the field `openBrackets` of a `Runs` object always holds the same array, whose elements and length may still change. The second is a typed array's length, the length $N$ of its view store. The third is an identity location. All three belong to the value class, and no admissible transition (Definition 6) changes them.

## 3 Model

Two notions carry the whole account. *Well-formed* is a property of one state. *Admissible* is a relation between two states. The rules of this note are statements about every admissible transition, and typed programs make only admissible transitions (Theorem 7).

**Definition 4** (World)**.** A *world* $W$ is a finite map from locations to pairs $(\kappa, T)$ of a kind $\kappa \in \{\mathsf{field}, \mathsf{cell}, \mathsf{elem}, \mathsf{store}, \mathsf{id}\}$ and a *contract* $T$; for a location of kind store the kind includes its form, array or $\mathsf{view}_N$. A contract is a type; it denotes a predicate on contents that is step-indexed and world-monotone [1]. A world $W'$ *extends* $W$, written $W \sqsubseteq W'$, when $\operatorname{dom} W \subseteq \operatorname{dom} W'$ and $W'(l) = W(l)$ for every $l \in \operatorname{dom} W$.

Example: for the heap of Section 2, $W(0) = (\mathsf{field}, \texttt{number})$, $W(3) = (\mathsf{store\ array}, \text{stores of number cells})$ and $W(6) = (\mathsf{id}, \{\texttt{null}\})$. After `a.push(40)` the world is $W$ with one more entry, $10 \mapsto (\mathsf{elem}, \texttt{number})$, and it extends $W$.

**Definition 5** (Well-formed state)**.** A *state* is a tuple $s = (\rho, h, W, k)$ of an environment, a heap, a world and a budget $k > 0$ of remaining steps. It is *well-formed* when:

**(W1) one domain** $\operatorname{dom} h = \operatorname{dom} W$.

**(W2) contents by kind** For every $l$ with $W(l) = (\kappa, T)$: $h(l)$ is a value when $\kappa \in \{\mathsf{field}, \mathsf{cell}, \mathsf{elem}\}$, a store when $\kappa = \mathsf{store}$, of length $N$ when its form is $\mathsf{view}_N$, and `null` when $\kappa = \mathsf{id}$.

**(W3) valid references** Every cell that a store of $h$ lists is a location of kind elem. For every object that occurs in $h$ or in $\rho$, each field location is of kind field or cell, its store location is of kind store, and its identity location is of kind id.

**(W4) contracts** $h(l)$ is within $T$ for every $l$ with $W(l) = (\kappa, T)$, and each value of $\rho$ meets its type, at the budget $k$.

(W1) to (W3) do not depend on the budget. From here on, *state* means well-formed state.

Example: the heap of Section 2 with its environment is a well-formed state. The same heap with $\langle 4, \circ, 12\rangle$ at location 3 is not: 12 is no location, against (W3). Neither is it with $\langle 4, \circ, 6\rangle$ there: 6 is an identity location.

A write within a location's contract preserves every typing judgment, whatever aliases exist [1]. This is the one use the note makes of types.

**Definition 6** (Admissible transition)**.** Let $s = (\rho, h, W, k)$ and $s' = (\rho', h', W', k')$ be states. $s'$ is *admissible from* $s$ when $W \sqsubseteq W'$ and $k' \le k$. We write $h \rightarrow_\rho h'$ when $h$ and $h'$ are the heaps of two such states and $\rho'$ extends $\rho$ [Lean: `Admissible`].

The definition mentions no program. Unfolded: every location of $h$ is a location of $h'$ with the same kind and the same contract, $h'$ holds contents of that kind within that contract, and $h'$ may have more locations. Example: in the heap of Section 2, the call `inc()` changes location 2 from 0 to 1; its contract is "a number", so the transition is admissible. A heap holding a string at location 2 is the heap of no state with this world, so no admissible transition from $h$ reaches it; nor one holding a number at location 3, whose kind is store.

**Theorem 7** (Preservation)**.** *Let $t$ be a term that the type system accepts at a state $s$. If $t$ runs from $s$ for $j < k$ steps and reaches a heap $h'$, then $h'$ is the heap of a state that is admissible from $s$, with budget $k - j$ [Lean: `run`, `run_normal`, `run_throw`, `safe_run` for (W4); `HasTyUsing.sound` and `ControlFlow.state_safe` for (W1) to (W3)].*

So what is proved below of every admissible transition holds of every step, call and callback of typed code. Host

and library code is modelled by declared contracts; that a model is accurate is audited, not proved.

The relation has four properties, used throughout:

**(T1) Preorder**
$h \to_\rho h$; and if $h \to_\rho h'$ and $h' \to_\rho h''$ then $h \to_\rho h''$, the worlds composing through the intermediate state's world [Lean: `Admissible.refl`, `Admissible.trans`].

**(T2) Growth**
If $h \to_\rho h'$ then $|h| \le |h'|$, and every location of $h$ has in $h'$ the kind and form it had: a transition allocates and never removes or re-purposes a location [Lean: `Admissible.length_le`].

**(T3) Fixed objects**
Let $v : \mathit{Key} \rightharpoonup \mathit{Loc}_{\mathsf{field}} \cup \mathit{Loc}_{\mathsf{cell}}$ be an object value and $h \to_\rho h'$. The domain of $v$ and each location $v(f)$ are the same before and after: the field $f$ of $v$ is read as $h(v(f))$ in $h$ and as $h'(v(f))$ in $h'$. This holds because $v$ is a value: it does not depend on the heap.

**(T4) Immutable locations**
If $h \to_\rho h'$, $l < |h|$ and the contract of $l$ admits the single value $u$, then $h(l) = h'(l) = u$ [Lean: `Admissible.stable_value`].

**Changes.** A *change* bounds how the contents of one location may differ between two heaps. One of the changes below, grow, must say that the cells a store newly lists are *new* locations. So a change takes a parameter $b$, a heap length: a location is new when it is at or above $b$. Formally a change $\delta$ is a family of preorders $\delta_b$ on contents, one for each $b \in \mathbb{N}$, with $\delta_{b'} \subseteq \delta_b$ for $b \le b'$: a larger $b$ leaves fewer locations new, so it allows less. We write $\delta_b(u, u')$ when $\delta_b$ relates the old contents $u$ to the new contents $u'$.

The action of an effect sets $b = |h|$ for its entry heap $h$. Only condition (iv) of grow depends on $b$. Write $N$ for the maximal array length. For contents $u, u'$:

$$
\begin{array}{lcl}
\mathsf{keep}_b(u,u') & \iff & u' = u \\
\mathsf{init}_b(u,u') & \iff & u' = u \ \lor \ (u \in \{\mathbf{null}, \mathbf{undefined}\} \land u' \in \mathit{Val}) \\
\mathsf{scalar}_b(u,u') & \iff & u' = u \ \lor \ (u \in \mathit{Val} \setminus \mathit{Obj} \land u' \in \mathit{Val}) \\
\mathsf{grow}_b(u,u') & \iff & \mathsf{scalar}_b(u,u') \ \lor \ (u, u' \in \mathit{Store} \land u \preceq_b u') \\
\mathsf{any}_b(u,u') & \iff & \mathsf{true}
\end{array}
$$

where, for two stores, $\sigma \preceq_b \sigma'$ (read: $\sigma'$ is a *growth* of $\sigma$ at $b$) is the conjunction of four conditions:

(i) $|\sigma| \le |\sigma'|$
(ii) $\forall n.\ \mathit{dense}_n(\sigma) \Rightarrow \mathit{dense}_n(\sigma')$, where $\mathit{dense}_n(\sigma) \iff n \le |\sigma| \land \forall i < n.\ \sigma_i \neq \circ$
(iii) $|\sigma| \le N \Rightarrow |\sigma'| \le N$
(iv) $\mathit{cells}(\sigma') \setminus \mathit{cells}(\sigma) \subseteq \{l \mid l \ge b\}$.

The change and the contract both bound the new contents. For example, $\mathsf{scalar}_b(1, \texttt{"a"})$ holds. A contract `number | string` admits this change; a contract `number` forbids it.

Each change allows all that the one before it allows:

$$\mathsf{keep} \sqsubseteq \mathsf{init} \sqsubseteq \mathsf{scalar} \sqsubseteq \mathsf{grow} \sqsubseteq \mathsf{any}.$$

[Lean: `Change.Holds`; `Change.Holds.refl`, `.mono`, `.trans`, `.lower`]

Examples. Let $l$ be the location of `c`, holding **`undefined`**. The assignment in `c ??= build()` changes the contents of $l$ from **`undefined`** to the value $v$ that `build()` returns, and $\mathsf{init}_b(\mathbf{undefined}, v)$ holds. The assignment `o.n = 3` changes the contents of the field location of `n` from 1 to 3, and $\mathsf{scalar}_b(1, 3)$ holds. With $b = 10$, $\mathsf{grow}_{10}$ relates the store $\langle 4, \circ, 5\rangle$ to $\langle 4, \circ, 5, 10\rangle$ (a `push`) and to $\langle 4, 10, 5\rangle$ (a write at the hole). It does not relate it to $\langle 4\rangle$, which fails (i), nor to $\langle 4, \circ, 5, 0\rangle$, which fails (iv): 0 is an old location.

**Regions.** Figure 1 shows the nonidentity locations of the heap of Section 2 and the portions of each region on those locations.

A *path* $\pi = x.f_1 \dots f_k$ is a variable followed by field names. Its *read* in $(\rho, h)$ starts from $\rho(x)$ and follows the fields $f_1, \dots, f_k$ in turn through $h$; the read *fails* when a value on the way is not an object with the next field. A *region* $r$ denotes a set $r(\rho, h)$ of locations of $h$. There are five forms:

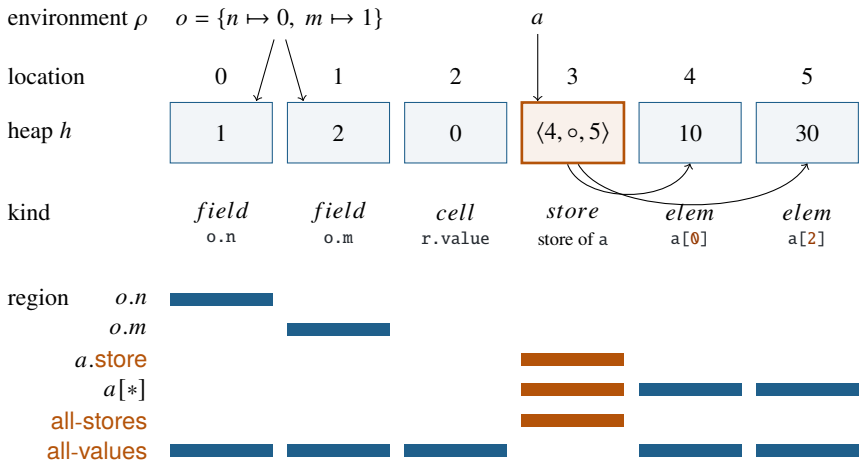



Figure 1: The nonidentity locations of the heap of Section 2 and the portions of each region on them. Identity locations 6–9 are omitted. Top: the object `o` points to its two field locations, and the array `a` to its store location. Middle: the six depicted locations with their contents and kinds; the store at location 3 lists the element cells 4 and 5, with a hole between them. Bottom: each bar marks a depicted location the region denotes; value locations are blue and store locations orange.

all-stores
: Every location of class store: all-stores$(\rho, h) = Loc_{\text{store}}$. In the heap of Section 2 it denotes $\{3\}$.

all-values
: Every location of class value: all-values$(\rho, h) = Loc_{\text{field}} \cup Loc_{\text{cell}} \cup Loc_{\text{elem}} \cup Loc_{\text{id}}$. In the heap of Section 2 it denotes $\{0, 1, 2, 4, 5, 6, 7, 8, 9\}$.

$\pi$.store
: The store location of the one array that $\pi$ reads: $(\pi.\text{store})(\rho, h) = \{s\}$, where $s$ is that array's store location. Example: $a$.store denotes $\{3\}$.

$\pi[*]$
: That store location together with the element cells its store lists: $(\pi[*])(\rho, h) = \{s\} \cup \mathit{cells}(h(s))$. Example: $a[*]$ denotes $\{3, 4, 5\}$.

$\pi.f$
: The field location for the key $f$ of the one object $v$ that $\pi$ reads: $(\pi.f)(\rho, h) = \{v(f)\}$. Examples: $o.n$ denotes $\{0\}$ and $o.m$ denotes $\{1\}$.

The first two regions are named for what they cover: all the stores of the heap, and all its value locations. In TScert's `Modifies` annotation they are written `stores` and `values`. A region at a path whose read fails denotes the empty set: if `z` is **`null`**, then $z.\mathit{open}$.store denotes $\emptyset$.

**Effects.** An *effect* says where a step may change the heap and by how much. It is a finite set of entries $r : \delta$, each a region and a change:

$$\begin{aligned} \Phi &::= \{ r_1 : \delta_1, \ \ldots, \ r_k : \delta_k \} \\ r &::= \text{all-stores} \mid \text{all-values} \mid \pi.\text{store} \mid \pi[*] \mid \pi.f \\ \delta &::= \text{keep} \mid \text{init} \mid \text{scalar} \mid \text{grow} \mid \text{any} \end{aligned}$$

Two examples. The effect {all-values : any, all-stores : grow} reads: every location that is not a store may change arbitrarily, and every array may grow and none may shrink. It is the effect of an element write `out[j] = v` whose target array is unknown. The effect {all-values : any, $y$.store : any} reads: every location that is not a store may change arbitrarily, the one array $y$ may change arbitrarily, and every other array is unchanged. It is the effect of a call that may shorten $y$, as `y.splice(...)` does.

**The action of an effect.** An effect $\Phi$ denotes a relation $\xrightarrow{\Phi}_\rho$ between heaps, its *action*. We say that $h'$ is reached from $h$ by a change *bounded by* $\Phi$ when $h \xrightarrow{\Phi}_\rho h'$:

$$h \xrightarrow{\Phi}_\rho h' \iff h \to_\rho h' \ \wedge\ \forall l < |h|.\ \big(h'(l) = h(l) \ \vee\ \exists (r : \delta) \in \Phi.\ l \in r(\rho, h) \wedge \delta_{|h|}(h(l), h'(l))\big).$$

Regions are read in the entry heap $h$, as in dynamic frames [9]. New locations are constrained by admissibility alone.

For example, let $h$ be the heap of Section 2 and let $h'$ be $h$ after `a.push(40)`: $h'(3) = \langle 4, \circ, 5, 10\rangle$, $h'(10) = 40$, and $h'$ agrees with $h$ on every location of $h$ but 3. Then $h \xrightarrow{\{\text{all-stores}:\text{grow}\}}_\rho h'$: location 3 is in all-stores$(\rho, h)$ and changed within $\text{grow}_{10}$, the other locations of $h$ are unchanged, and 10 is new. After `a.length = 1` the store is $\langle 4\rangle$, a change that is not within grow; that transition is bounded by $\{a.\text{store} : \text{any}\}$ and is not bounded by $\{\text{all-stores} : \text{grow}\}$.

**Order, top, rigidity.** We write $\Phi \sqsubseteq \Psi$ when the action of $\Phi$ is included in the action of $\Psi$, for every $\rho$. We write $\top$ for $\{\text{all-values} : \text{any},\ \text{all-stores} : \text{any}\}$, the effect of a step about which nothing is known; every effect is below $\top$.

An effect $\Phi$ is *rigid* when its action is transitive: if $h \xrightarrow{\Phi}_\rho h'$ and $h' \xrightarrow{\Phi}_\rho h''$ then $h \xrightarrow{\Phi}_\rho h''$. Rigidity is what lets a fact that survives one change bounded by $\Phi$ survive any number of them.

**Rigid: class regions**
An effect whose regions are all-stores and all-values is rigid at every named change. A change below any keeps each location in its class, and any absorbs any change that follows it.

**Rigid: a store at a rigid path**
$\{\pi.\text{store} : \delta\}$ is rigid on the states where $\pi$ is rigid (Section 4): the path then reads the same store in $h$ and in $h'$.

**Not rigid: cells of a store changed at any**
$\{x[*] : \text{any}\}$ is not rigid. A first change may make the store of $x$ list an element cell that existed before and that it did not list, such as a cell $l$ of another array. A second change, whose region is read in the new heap, may then change $l$, which the region read in $h$ does not contain.

**The effect of a command.** The judgment $c : \Phi \mid \Phi^e$ states two bounds on a command $c$ run from $(\rho, h)$. If $c$ returns, in a heap $h'$, then $h \xrightarrow{\Phi}_\rho h'$. If $c$ throws, in a heap $h'$, then $h \xrightarrow{\Phi^e}_\rho h'$. We write $c : \Phi$ when $\Phi^e$ is $\top$. For example `out[j] = v` : {all-values : any, all-stores : grow}. This judgment is what a `Modifies` annotation states and what an effect system derives [Lean: `Post`; for graph code `run_sound_single_pass`, whose guarantee states the bound over class regions and the regional bound of an accepted function].

# 4 The assertion language

**Syntax.** Assertions $F, G$ are built from atoms $a$:

| | | | |
|---|---|---|---|
| $F, G$ | ::= | $a \mid F \wedge G \mid g \vee F \mid \exists v.\ F$ | $g$ an invariant assertion; $v$ a logical variable |
| | $\mid$ | $\pi \,\#\, \pi' \mid \text{apart}(x, y) \mid \text{fresh}(x)$ | separation and freshness |
| | $\mid$ | $\Box_\Phi F$ | the modality |

**Atoms.** An atom is a fact of a first-order language over the heap. An atom is true only where every path it reads is defined. There are five forms:

**Linear inequality**
An inequality between linear terms over variables and lengths $|\pi|$ of arrays at paths. Examples: $i < |\mathit{flags}|$ and $\mathit{to} \le |\mathit{xs}|$.

**Equality** $\pi \doteq v$
The path $\pi$ reads the value $v$. Example: $z \doteq$ `null`.

**Density** dense$(x, n)$
The first $n$ indices of the array $x$ are present. For `a = [10, , 30]`, dense$(a, 1)$ holds and dense$(a, 2)$ does not.

**Kind of contents**

The contents of a cell are of a stated kind, such as a number or an object. Example: the contents of $x.f$ are a number.

**Bounded quantification** $\mathsf{every}(z, t, \varphi)$

Each element of the array $z$ at an index below $t$ satisfies $\varphi$. For `a = [10, , 30]`, $\mathsf{every}(a, 1, v > 0)$ holds.

**Satisfaction.** An assertion is interpreted at a triple $(\rho, m, h)$: an environment, an *entry mark*, and a heap. The entry mark $m \le |h|$ is the length the heap had when the current function invocation started; the locations at or above $m$ are the ones the invocation has allocated. We write $\rho, m, h \models F$. Atoms read $\rho$ and $h$ only, and the connectives $\wedge$, $\vee$, $\exists$ have their usual meaning. The other four forms are:

$$\begin{array}{lcl}
\rho, m, h \models \pi \mathbin{\#} \pi' & \iff & \pi \text{ and } \pi' \text{ read arrays with different store locations,} \\
\rho, m, h \models \mathsf{apart}(x, y) & \iff & x \mathbin{\#} y, \text{ and the stores of } x \text{ and } y \text{ list no common element cell,} \\
\rho, m, h \models \mathsf{fresh}(x) & \iff & \text{the store location of } \rho(x) \text{ is at or above } m, \\
\rho, m, h \models \Box_\Phi F & \iff & \text{for all } h', \text{ if } h \xrightarrow{\Phi}_\rho h' \text{ then } \rho, m, h' \models F.
\end{array}$$

So $\Box_\Phi F$ says: $F$ holds in every heap reached from here by a change bounded by $\Phi$. Since $h \xrightarrow{\Phi}_\rho h$, it says in particular that $F$ holds here.

Two examples, in the heap $h$ of Section 2, where $|a| = 3$. First, $\rho, m, h \models \Box_{\text{all-values:any, all-stores:grow}}(3 \le |a|)$: a change bounded by this effect leaves the store of $a$ or grows it, so its length stays at least 3. Second, $\rho, m, h \not\models \Box_{a.\text{store:any}}(3 \le |a|)$: the heap after `a.length = 1` is reached by a change bounded by this effect, and there $|a| = 1$. In a function `f() { const xs = [1]; ...}`, $\mathsf{fresh}(xs)$ holds after the declaration: the store of `xs` was allocated by this invocation.

Assertions describe the whole heap. They can be duplicated, and holding one confers no permission to write.

**Invariant assertions and rigid paths.** Two abbreviations:

$$F \text{ is } \textit{invariant} \triangleq F \vdash \Box_\top F \qquad \mathsf{rigid}(\pi) \triangleq \exists v.\, \Box_\top(\pi \doteq v).$$

An invariant assertion, once true, is true after every admissible transition. A fact about variables alone, such as $i < n$, is invariant. So is $x \mathbin{\#} y$ for two variables: an array's store location is fixed when the array is allocated. A path is rigid when it reads the same value after every admissible transition. A path through fields whose contracts admit one value is rigid: `runs.openBrackets` is rigid when `openBrackets` is declared `@stable`. In $\mathsf{rigid}(\pi)$ the witness $v$ is a logical variable, so it is the same value before and after the change.

> *Remark* 8 (Reading). $\Box_\Phi F$ is the weakest liberal precondition of $F$ for the command that makes one arbitrary admissible change bounded by $\Phi$: $\Box_\Phi F = \mathit{wlp}(\mathsf{havoc}\,\Phi, F)$, the reading a verifier gives a call when it replaces it by its `modifies` clause. For a rigid $\Phi$ it is the weakest assertion that is stronger than $F$ and stable under $\xrightarrow{\Phi}$: the floor of explicit stabilisation [11], at a rely denoted by an effect.

> **Proposition 9** (Laws). *[Lean: box_elim, box_and, box_mono, box_anti, box_or, box_idem]*
>
> *(a)* $\Box_\Phi F \vdash F$.
>
> *(b)* $\Box_\Phi(F \wedge G) \dashv\vdash \Box_\Phi F \wedge \Box_\Phi G$; *if* $F \vdash G$ *then* $\Box_\Phi F \vdash \Box_\Phi G$.
>
> *(c)* *If* $\Phi \sqsubseteq \Psi$ *then* $\Box_\Psi F \vdash \Box_\Phi F$.
>
> *(d)* $g \vee \Box_\Phi F \vdash \Box_\Phi(g \vee F)$ *for invariant* $g$.
>
> *(e)* $\Box_\Phi F \vdash \Box_\Phi \Box_\Phi F$ *for rigid* $\Phi$.

Law (a) follows from reflexivity; (b), (c) and (d) from the definitions; (e) from transitivity. For class regions, transitivity is `Closed.Holds.trans_on`; for a store at a path it is `ArgumentStoreRegionsWithin.trans`, provided the path resolves to the same store in both heaps.

# 5 Entering the modality

A fact enters the modality by naming what it reads and how much change of it the fact tolerates.

**Definition 10** (Support)**.** A *support* for an assertion $F$ is a finite set $S$ of pairs $r : \delta$, a region and a change, with this property: for all $\rho, m, h, h'$, if $\rho, m, h \models F$, $h \to_\rho h'$, and every location $l < |h|$ in a region $r(\rho, h)$ of a pair $(r : \delta) \in S$ satisfies $\delta_{|h|}(h(l), h'(l))$, then $\rho, m, h' \models F$. Locations outside the regions of $S$ may change in any admissible way.

For example $\{x.\mathsf{store} : \mathsf{grow}\}$ supports $n \le |x|$: the fact reads the store of $x$ and nothing else in the heap, and it tolerates any growth of that store.

**Definition 11** (Respect)**.** Let $K$ be an assertion. An effect $\Phi$ *respects* a support $S$ under $K$, written $\Phi \triangleleft_K S$, when for every pair $(r : \delta) \in S$ and every entry $(r' : \gamma) \in \Phi$, either $\gamma \sqsubseteq \delta$, or $K$ entails that $r$ and $r'$ denote disjoint sets.

For example $\{\mathsf{all\text{-}values} : \mathsf{any},\ \mathsf{all\text{-}stores} : \mathsf{grow}\}$ respects $\{x.\mathsf{store} : \mathsf{grow}\}$ under $\mathsf{true}$. The entry $\mathsf{all\text{-}stores} : \mathsf{grow}$ meets the first condition, $\mathsf{grow} \sqsubseteq \mathsf{grow}$. The entry $\mathsf{all\text{-}values} : \mathsf{any}$ meets the second: a store location is no location of class value.

**Theorem 12** (Tolerant frame)**.** *If $S$ supports $F$ and $\Phi \triangleleft_K S$, then $K \wedge F \vdash \Box_\Phi F$. [Lean: `tolerant_frame`, from `stable_sound` and `frame_sound`, for five forms of fact with their supports: a cell's contents, a link of a variable to a field, a value's kind, density, and the kind of elements. Linear facts enter the modality by the axioms below]*

*Proof.* Let $h \xrightarrow{\Phi}_\rho h'$ and $l \in r(\rho, h)$ with $(r : \delta) \in S$. Either $h'(l) = h(l)$, and $\delta$ is reflexive, or some $(r' : \gamma) \in \Phi$ covers $l$ and bounds its change by $\gamma$. Then $r$ and $r'$ are not disjoint, so $\gamma \sqsubseteq \delta$. □

The theorem has two known rules as its extreme cases. When every change in the support is $\mathsf{keep}$, the condition is that the regions a command writes are disjoint from the regions a formula reads: the frame rule of region logic [4]. When $K$ separates no pair, the condition is that the assertion is stable under an interfering action [8]. The theorem chooses between the two at each pair.

A fact may have several supports and none of them greatest. The fact $\mathsf{apart}(x, y)$ is supported by $\{x[*] : \mathsf{grow},\ y[*] : \mathsf{keep}\}$: a store that grows lists only the cells it listed and new ones. It is also supported by the mirror image, $\{x[*] : \mathsf{keep},\ y[*] : \mathsf{grow}\}$. It is not supported by $\{x[*] : \mathsf{grow},\ y[*] : \mathsf{grow}\}$: both stores may come to list the same new cell.

**Proposition 13** (Axioms of the atoms)**.** *For variables $x, y, z, w$, a rigid effect in (Rigid), and a an atom that reads, besides variables, only what is displayed:*

| | |
|---|---|
| *(Grow)* | $n \le \lvert x\rvert \ \vdash\ \Box_{\mathsf{all\text{-}values}:\mathsf{any},\ \mathsf{all\text{-}stores}:\mathsf{grow}}\ (n \le \lvert x\rvert)$ |
| *(Dense)* | $\mathsf{dense}(x, n) \ \vdash\ \Box_{\mathsf{all\text{-}values}:\mathsf{any},\ \mathsf{all\text{-}stores}:\mathsf{grow}}\ \mathsf{dense}(x, n)$ |
| *(Sep)* | $x \# y \wedge a(\lvert x\rvert) \ \vdash\ \Box_{\mathsf{all\text{-}values}:\mathsf{any},\ y.\mathsf{store}:\mathsf{any}}\ a(\lvert x\rvert)$ |
| *(Every)* | $\mathsf{apart}(x, z) \wedge \mathsf{every}(z, t, \varphi) \ \vdash\ \Box_{x[*]:\mathsf{any}}\ \mathsf{every}(z, t, \varphi)$ *t a variable, $\varphi$ over scalars* |
| *(Init)* | $a(\lvert\pi\rvert) \ \vdash\ \Box_{\mathsf{all\text{-}values}:\mathsf{init}}\ a(\lvert\pi\rvert)$ |
| *(Rigid)* | $\mathsf{rigid}(\pi) \wedge a(\lvert\pi\rvert) \ \vdash\ \Box_\Phi\, a(\lvert\pi\rvert)$ *if $a(\lvert y\rvert) \vdash \Box_\Phi\, a(\lvert y\rvert)$ for a fresh $y$* |
| *(View)* | $a(\mathsf{view}(w)) \ \vdash\ \Box_\top\, a(\mathsf{view}(w))$ $\mathsf{view}(w)$ *a typed array's length* |

The marks below give each axiom's scope.

**(Grow)**
A lower bound on a length survives any change that only grows arrays. It mentions no separation: it is Example 1. Lean: `grow`.

**(Dense)**
A dense prefix survives any change that only grows arrays. Lean: `dense_support`, the statement that $\{x.\mathsf{store} : \mathsf{grow}\}$ supports $\mathsf{dense}(x, n)$; the axiom follows by Theorem 12.

**(Sep)**
A fact about the length of $x$ survives an arbitrary change of another array $y$. It is Example 2. [conj.] Lean: `regionFirst_kept`, for the store of an array a variable holds, across a call with a regional effect.

**(Every)**
A fact about each element of $z$ survives an arbitrary change of an array $x$ that shares no element cell with $z$. [conj.] Lean: `writeAtoms_holds`, for an element write at a checked index.

**(Init)**
A fact about a length survives a change that only initialises: init changes no location that holds an object, a number or a store, and a defined read of a path passes only fields that hold objects. Lean: `init_box`, `init_path`, for linear facts over integers, array lengths and typed-array lengths at any paths.

**(Rigid)**
A fact that holds of an array named by a variable holds of the array at a rigid path. The substitution of the witness for $y$ is capture-avoiding. [conj.] as a rule of substitution. Lean: a path through fields that each type states stable is rigid (`stableAt_of_stableField`, `stableAt_cons`, `stable_rigid`), and the instance for (Grow) is `grow_path_typed`.

**(View)**
A fact about a typed array's length survives every change: that length is an immutable location, by (T4). Lean: `view_box`, `viewLengthOf_kept`.

# 6 Programs

**Triples.** A triple $\{P\}\ c\ \{Q \mid Q^e\}$ states partial correctness of a command $c$. If $c$ runs from a state satisfying $P$ and returns, the state it returns in satisfies $Q$. If it throws, the state at its handler satisfies $Q^e$. We write $\{P\}\ c\ \{Q\}$ when $Q^e$ is $\mathsf{true}$. For example $\{\mathsf{true}\}\ x = [\,]\ \{|x| = 0\}$.

The posts $Q$ and $Q^e$ are *two-state* assertions. For entry state $(\rho_0, h_0)$ and exit state $(\rho_1, h_1)$, $\mathsf{old}(B)$ means $\rho_0, m, h_0 \models B$; $m$ is the enclosing invocation's entry mark. Ordinary assertions use $(\rho_1, m, h_1)$. An old refers to the command whose triple contains it. For example, `x.push(v)` has post $|x| = \mathsf{old}(|x|) + 1$.

The variable that a step binds is fresh (Section 2). So an assertion $R$ over the variables in scope before $c$ is unaffected by the binding.

**Rules.**

$$\textsc{Entry}\ \frac{\{P\}\ c\ \{Q \mid Q^e\} \qquad c : \Phi \mid \Phi^e}{\{P \wedge B\}\ c\ \{Q \wedge \mathsf{old}(B) \wedge \mathsf{tr}_\Phi \mid Q^e \wedge \mathsf{old}(B) \wedge \mathsf{tr}_{\Phi^e}\}}$$

$$\textsc{Frame}\ \frac{\{P\}\ c\ \{Q \mid Q^e\} \qquad c : \Phi \mid \Phi^e}{\{P \wedge \square_\Phi R \wedge \square_{\Phi^e} R^e\}\ c\ \{Q \wedge R \mid Q^e \wedge R^e\}} \qquad \textsc{Frame}^+\ \frac{\{P\}\ c\ \{Q\} \qquad c : \Phi \qquad \Phi\ \text{rigid}}{\{P \wedge \square_\Phi R\}\ c\ \{Q \wedge \square_\Phi R\}}$$

$$\textsc{Loop}\ \frac{\{I\}\ c\ \{I\} \qquad c : \Phi \qquad \Phi\ \text{rigid}}{\{I \wedge \square_\Phi R\}\ \mathbf{loop}\ c\ \{I \wedge \square_\Phi R\}} \qquad \textsc{Call}\ \frac{\{P\}\ y = f(\vec{p})\ \{Q\} \qquad y = f(\vec{p}) : \Phi}{\{P[\vec{x}/\vec{p}]\}\ y = f(\vec{x})\ \{Q[\vec{x}/\vec{p}]\} \qquad y = f(\vec{x}) : \Phi[\vec{x}/\vec{p}]}$$

$$\textsc{Alloc}\ \frac{x\ \text{fresh} \qquad y\ \text{ranges over the arrays in scope}}{\{\mathsf{true}\}\ x = [\,]\ \{\mathsf{fresh}(x) \wedge |x| = 0 \wedge \textstyle\bigwedge_y \mathsf{apart}(x, y)\}}$$

$$\textsc{Seq-Eff}\ \frac{c_1 : \Phi \qquad c_2 : \Psi \qquad \Psi \rightsquigarrow_{ev} \Psi' \qquad \Phi \cup \Psi' \sqsubseteq \Phi}{c_1; c_2 : \Phi}$$

$$\textsc{Fresh-Eff}\ \frac{c : \Phi \cup \{x.\mathsf{store} : \delta\} \qquad \mathsf{fresh}(x)\ \text{holds before}\ c}{c : \Phi\ \ \text{from the entry}}$$

The rules one by one:

**Entry**
The assertion $\mathsf{tr}_\Phi$ means $h_0 \xrightarrow{\Phi}_{\rho_0} h_1$, using the action of Section 3. The precondition supplies $\mathsf{old}(B)$; the effect judgment supplies the action at each exit. For example, $|z| = j$ before a command and its post $|z| = \mathsf{old}(|z|) + 10$ imply $|z| = j + 10$. Lean: `compose_holds`, `Composer.fact_holds`, for a call's normal exit and the facts of TScert's checker, without $\mathsf{tr}_\Phi$; a conjecture for an arbitrary $B$.

**FRAME**
The frame rule, with $\wedge\,\square_\Phi$ where separation logic has $*$. It is ENTRY with $B = \square_\Phi R \wedge \square_{\Phi^e} R^e$: by the modality's definition, $\mathsf{old}(\square_\Phi R) \wedge \mathsf{tr}_\Phi \vdash R$, and the exceptional exit uses $\Phi^e$ and $R^e$. The effect is stated at the command's entry. Bound variables are renamed apart from the frame assertions and their effects.

**FRAME$^+$**
For a rigid effect the frame keeps its modality after $c$, by law (e).

**LOOP**
Induction on normal iterations uses FRAME$^+$. Here $c$ includes the guard evaluation, and every normal exit satisfies $I$. Loop-carried variables occur in $I$; neither $R$ nor $\Phi$ mentions them.

**CALL**
Instantiate the callee's specification at the actual arguments by capture-avoiding substitution; $Q$ may mention the result $y$. The displayed rule uses assertions independent of the invocation mark. Freshness assertions require rebasing between the callee's and caller's marks. Exceptional posts and effects instantiate in the same way; their defaults are $\mathsf{true}$ and $\top$. Lean: `call_rule`, `invocation_throw_action`.

**ALLOC**
A new array is fresh, empty, and apart from every array $y$ that exists before the step. These separations are facts of the state after the step. Later steps keep or lose them like any other fact: a growth of the new array's store keeps them, and an arbitrary change of that store does not, since it may make the store list a cell of $y$. Lean: `alloc_post`, and `alloc_apart_typed` for arrays of an array type; `apart_grow_keep`, `apart_not_kept_fresh`.

**SEQ-EFF**
The effect of a sequence, stated at the entry of the sequence. The effect $\Psi$ of the second command is read where that command starts, so it is first *rebased* to the entry (below).

**FRESH-EFF**
A change to the store of an array that the command itself allocated is no part of the command's effect, seen from its entry. A fresh store may list a cell of the entry heap, and a change of that cell is part of the effect.

> **Definition 14** (Rebasing)**.** Let $\Psi$ be an effect read at an intermediate heap and $\Psi'$ an effect read at the entry heap. $\Psi \rightsquigarrow_{ev} \Psi'$ holds when every location of the entry heap that $\Psi$ covers at a change $\delta$ is covered by $\Psi'$ at $\delta$ or at a wider change. The evidence $ev$ records, for each value that $\Psi$ names, the entry path it was read from, and that this path reads the same in both heaps.

Three cases follow from the definition. A region at a rigid path rebases to itself. The store of a fresh array is no location of the entry heap, so its entry rebases to nothing: this is FRESH-EFF. Any other region, and the cells of a fresh store, widen to their class. For example, in `f(o) { const xs = o.items; xs.push(1); }` the push has the effect $\{xs.\mathsf{store} : \mathsf{grow}\}$ where it starts. If $o.items$ is rigid, the effect of `f` at its entry is $\{o.items.\mathsf{store} : \mathsf{grow}\}$. Otherwise it widens to $\{\mathsf{all\text{-}stores} : \mathsf{grow}\}$ [Lean: `seq_eff`, `fresh_eff`, `fresh_cells_not_erased`, `rebaseAll`, `rebase_sound`, `rebaseAll_labels`. The Lean rebasing widens element cells and regions it cannot place to every location (`rebase_elems`), a wider bound than their class].

> **Example 15** (Freshness bounds effects, not facts)**.** Let `shrink(xs)` pop its argument, `shrink` : $\{\mathsf{all\text{-}values} : \mathsf{any}, xs.\mathsf{store} : \mathsf{any}\}$, and `f() { const xs = [1]; shrink(xs); return xs[0]; }`. By FRESH-EFF, `f` : $\{\mathsf{all\text{-}values} : \mathsf{any}\}$: a caller of `f` sees no change to any array. Inside `f` the fact $0 < |xs|$ is not derivable under $\square_{xs.\mathsf{store}:\mathsf{any}}$, so it does not hold after the call `shrink(xs)`, and the read `xs[0]` is refused. Freshness gives separations, by ALLOC. It does not put any fact under a modality.

> **Proposition 16** (Reliance)**.** *Let every native step and every callback have effect bounded by a rigid $\Phi$. If $\square_\Phi F$ holds initially, $F$ holds at every observation between these steps and calls. [Lean: `rely_chain`; `map_rely`, `elems_rely` for `map`]*

For example take `xs.map(f)`, where `map` reads $n = |xs|$ once at its start. Let $F = \mathsf{dense}(xs, n)$ and $\Phi = \{\mathsf{all\text{-}values} : \mathsf{any}, \mathsf{all\text{-}stores} : \mathsf{grow}\}$. If `f`, with all it calls, has an effect bounded by $\Phi$, then by (DENSE) and the proposition $F$ holds each time `map` reads an element. So the result of `xs.map(f)` is dense when `xs` is.

**Exact posts.** The post $|x| = \mathsf{old}(|x|) + 1$ of `x.push(v)` specifies a strong update on shared state. If $y$ may alias $x$, (Grow) preserves $n \le |y|$. The equality $|y| = n$ can become false when $y = x$. Entry combines these facts with the post.

# 7 Transfer along edges

A node of a control-flow graph has one *exit edge* for each way it completes: a normal return, each branch it can take, and a throw to a handler. An exit edge $e$ carries four things:

**The effect $\Phi_e$**
The effect of the node when it completes through $e$.

**The post $Q_e$**
A sound two-state post of the node on that exit. It includes the binding of the result, or of the thrown value.

**The condition $B_e$**
The condition under which a branch is taken; $\mathsf{true}$ on other edges.

**The renaming $\theta_e$**
A map from target variables to their source variables on $e$; it induces the target environment.

In Example 19 below, the true exit of the test `j < to` has $\Phi_e = \emptyset$, $B_e = (j < to)$ and the identity renaming; the back edge has $\theta_e = [j \mapsto j']$.

The *knowledge* $\Sigma$ at a point of the graph is a finite set of assertions. A state $s$ satisfies it at an entry mark $m$, written $s \models_m \Sigma$, when it satisfies each of them.

**Definition 17** (Checked transfer)**.** $\mathrm{Check}(c, \Sigma, e, A)$ holds when there is a derivation of

$$\mathsf{old}(\Sigma) \ \wedge \ \mathsf{tr}_{\Phi_e} \ \wedge \ Q_e \ \wedge \ B_e \ \vdash \ \theta_e^* A,$$

where $\mathsf{old}(\Sigma)$ is $\Sigma$ at the node's entry state, $\theta_e^* A$ is $A$ with the target's variables replaced by the node's, and the entailment includes that every path $A$ reads is defined.

**Theorem 18** (Transfer)**.** *[Lean: `transfer`, one theorem per exit, for exit effects over class regions; `graph_safe` composes the exits into the safety of a graph, and `loop_safe` is the loop of Section 7 as an instance. A conjecture for exit effects with regions at paths] If $s \models_m \Sigma$, the node $c$ runs from $s$ through exit $e$ to $s'$, and* $\mathrm{Check}(c, \Sigma, e, A)$*, then $\theta_e(s') \models_m A$.*

The proof is Entry with $B = \bigwedge \Sigma$, and consequence.

**Sound graphs.** Annotate each point of a graph with knowledge. The annotated graph is *sound* when its entry state satisfies the knowledge at the entry, and Check holds on every edge against the knowledge at the edge's target. Merges and loop heads are targets like any other.

**The forward pass.** A forward pass proposes knowledge for Check. A merge retains facts established by every incoming edge. Loop-head knowledge is supplied.

Derivations use ordinary two-state logic, linear arithmetic, and Propositions 9 and 13. They include old-state lifting: $F \vdash G$ gives $\mathsf{old}(F) \vdash \mathsf{old}(G)$. Modal elimination is the entailment in Frame. This system is sound and incomplete for semantic entailment.

**Example 19** (Example 1, in full)**.** With the loop variable a parameter of the head, and $\Sigma_H(j) = \{0 \le j, \ to \le$

$|xs|$}:

| | node or edge | effect | obligation and its reason |
|---|---|---|---|
| 1 | entry $\to H(from)$ | | $0 \le from,\ to \le |xs|$ given |
| 2 | $H(j)$ : `j < to` true | $\emptyset$ | $B = (j < to)$; $\Sigma_H$ by preservation of existing locations |
| 3 | $v$ = `xs[j]` | $\emptyset$ | the read: $0 \le j < to \le |xs|$ |
| 4 | `out[j] = v` | all-values:any, all-stores:grow | $to \le |xs|$ by (Grow); $0 \le j,\ j < to$ invariant |
| 5 | $j' = j + 1$ | $\emptyset$ | $Q = (j' = j + 1)$ |
| 6 | back edge, $\theta = [j \mapsto j']$ | | $\theta^*\Sigma_H = \{0 \le j',\ to \le |xs|\}$ from $0 \le j$, $Q$, row 4 |
| 7 | $H(j)$ : `j < to` false | $\emptyset$ | exit with $\Sigma_H \wedge \neg(j < to)$ |

No row mentions whether `out` is an alias of `xs`.
Suppose the invariant also includes $|xs| \le n$. The effect in row 4 permits growth beyond $n$, so that effect alone does not preserve the bound. The actual write is in bounds by row 3; its exact semantics preserves the length. The regional effect {all-values : any, $out$.store : grow} preserves the upper bound under $out \# xs$, by (Sep) and law (c). This separation is an obligation of rows 1 and 6.

# 8 The patterns

| | | |
|---|---|---|
| Lower bound under aliasing (Ex. 1) | $to \le |xs| \vdash \square_{\text{all-values:any, all-stores:grow}}(to \le |xs|)$ | (Grow) |
| Bound across a rewriting call (Ex. 2) | $flags \# y \wedge i < |flags| \vdash \square_{\text{all-values:any, } y\text{.store:any}}(i < |flags|)$, $y$ standing for the rigid *runs.openBrackets* | (Sep), (Rigid) |
| Fresh array store | its writes are omitted from the invocation's entry effect; apart$(x, y)$ for older $y$ | Fresh-Eff, Alloc |
| Write-once cache, `c ??= build()` | $a \vdash \square_{\Phi_{build} \cup \{\text{all-values:init}\}} a$, by Theorem 12 at a support respected by $\Phi_{build}$ and tolerating init | (Init) |
| Reliance during callbacks | dense$(xs, n) \vdash \square_{\text{all-values:any, all-stores:grow}}$dense$(xs, n)$ | Prop. 16 |
| Quantified fact across a write | apart$(a, z) \wedge$ every$(z, t, \varphi) \vdash \square_{a[*]\text{:any}}$every$(z, t, \varphi)$ | (Every) |
| Structures built in step | $\{|a| = |b| \wedge a \# b\}$ `a.push(v)` $\{|a| =$ old$(|a|) + 1 \wedge |b| =$ old$(|b|)\}$ post, (Sep) | |
| Separation where a value exists | $(z \doteq$ **`null`** $\vee\ xs \# z.open) \wedge a(|xs|) \vdash \square_{\text{all-values:any, } z.open\text{.store:any}}\, a(|xs|)$; at **`null`** the region is empty | |
| Fact through a field | rigid$(x.f) \wedge n \le |x.f| \vdash \square_{\text{all-values:any, all-stores:grow}}(n \le |x.f|)$; fails without rigid | (Rigid) |
| Typed-array length | $i <$ view$(w) \vdash \square_{\top}(i <$ view$(w))$ | (View) |
| Fact at a handler | $\square_{\Phi^e} R^e$ | Frame |

Four patterns are outside the language. Three need a form it lacks:

### A fact about a field of each element

Example: `rows[i].n` $\ge 0$ for every $i$. It needs a region $z[*].n$: a path from each element cell the store lists.

### Overlapping typed-array views

Two views are distinct objects over one buffer, so # on their stores says nothing about their elements. They need regions of buffer cells with offsets. TScert's model states a write through a view without them, by the forms of stores. The write's post reads the two states and the entry world: every location that is not a view store in the entry world keeps its contents, and a view store may come to list new cells, which hold what its typed array then reads. So facts about values, lengths and ordinary arrays survive the write, and a fact about the elements of any typed array does not [Lean: `ViewWrite`, `Effect.viewPost`].

### A callback that writes what it captured

Example: `walk(xs, v => out.push(v))`. Each use has a closed effect. A reusable specification of `walk` needs an effect variable, $\forall\varepsilon.\ \Phi \cup \varepsilon$.

The fourth needs no new form. It needs a separation that the rules do not establish:

### A record replaced on each loop pass

The new record must be separated from the loop's iterator on every back edge. Alloc states separation only against what is in scope at the allocation.

# 9 Relation to separation logic

The language takes from separation logic [10] the frame rule and, in Alloc and Fresh-Eff, the principle that what a computation allocates is outside every frame of its callers. It has no separating conjunction and no resource model: assertions describe the whole heap and are duplicable. In Frame the effect is stated, where separation logic derives a command's footprint from its precondition; in return the modality frames facts that overlap what the command writes. A precise correspondence would embed assertions about a subheap into assertions about the whole heap and relate $P * R$ to $\Box_\Phi R$ for the effect $\Phi$ confined to $P$'s subheap; we state none. Whether the model is an instance of abstract separation logic [6] or of Views [7] is open; it would need a composition on supports.

The closest relatives are region logic and dynamic frames, for effects as region expressions read in the pre-state; rely/guarantee with explicit stabilisation, for the modality; and monotone state [2, 3], for init and grow, which here are enforced by the effect judgment and by no resource discipline.

**How much separation.** In these patterns: pairwise disjointness of single stores (#), and of a store with its cells (apart) where a fact reads element contents, each established by allocation or assumed at a function's entry. Five of the eleven obligations use none. This describes the patterns met so far and is no completeness claim.

# 10 Limits

### Effects are closed

An effect has no variables; Section 8 states the consequence for callbacks [5].

### Await

Across an `await` the effect is that of the environment, $\top$.

### Broken and restored invariants

Triples express restored assertions; admissibility (Section 3) governs intermediate states.

### Fixed contracts

World extension (Section 3) excludes type-changing updates.

### Transfer at regional effects

Theorem 18 states the Lean scope.

# References


[1] A. Ahmed. *Semantics of Types for Mutable State*. PhD thesis, Princeton University, 2004.

[2] A. Ahmed, D. Dreyer, A. Rossberg. State-dependent representation independence. POPL 2009.

[3] D. Ahman, C. Fournet, C. Hriţcu, K. Maillard, A. Rastogi, N. Swamy. Recalling a witness: foundations and applications of monotonic state. PACMPL 2(POPL), 2018.

[4] A. Banerjee, D. Naumann, S. Rosenberg. Local reasoning for global invariants, part I: region logic. J. ACM 60(3), 2013.

[5] L. Birkedal, N. Torp-Smith, H. Yang. Semantics of separation-logic typing and higher-order frame rules. LICS 2005.

[6] C. Calcagno, P. O'Hearn, H. Yang. Local action and abstract separation logic. LICS 2007.

[7] T. Dinsdale-Young, L. Birkedal, P. Gardner, M. Parkinson, H. Yang. Views: compositional reasoning for concurrent programs. POPL 2013.

[8] C. B. Jones. Tentative steps toward a development method for interfering programs. ACM TOPLAS 5(4), 1983.

[9] I. Kassios. Dynamic frames: support for framing, dependencies and sharing without restrictions. FM 2006.

[10] P. O'Hearn, J. Reynolds, H. Yang. Local reasoning about programs that alter data structures. CSL 2001.

[11] J. Wickerson, M. Dodds, M. Parkinson. Explicit stabilisation for modular rely-guarantee reasoning. ESOP 2010.